

開始使用 gRPC-Go - 串流

來源：<https://codelabs.developers.google.com/grpc/getting-started-grpc-go-streaming?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何使用 gRPC-Go 建構 gRPC 串流服務。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 定義訊息和服務](#)
4. [4. 產生用戶端和伺服器程式碼](#)
5. [5. 實作服務](#)
6. [6. 建立用戶端](#)
7. [7. 立即試用](#)
8. [8. 後續步驟](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將使用 gRPC-Go 建立用戶端和伺服器，做為以 Go 編寫的路線對應應用程式基礎。

完成本教學課程後，您將擁有一個用戶端，可使用 [gRPC](#) 連線至遠端伺服器，取得用戶端路線上的功能資訊、建立用戶端路線摘要，以及與伺服器和其他用戶端交換路線資訊 (例如交通資訊更新)。

服務定義於 Protocol Buffers 檔案中，用於產生用戶端和伺服器的樣板程式碼，以便彼此通訊，節省您實作該功能的時間和精力。

產生的程式碼不僅會處理伺服器與用戶端之間複雜的通訊，也會處理資料序列化和還原序列化。

課程內容

- 如何使用通訊協定緩衝區定義服務 API。
- 如何使用自動程式碼生成功能，從通訊協定緩衝區定義建構以 gRPC 為基礎的用戶端和伺服器。
- 瞭解 gRPC 的用戶端與伺服器串流通訊。

這個程式碼研究室適合剛接觸 gRPC 的 Go 開發人員、想複習 gRPC 的使用者，以及有興趣建構分散式系統的任何人。不需要有 gRPC 相關經驗。

步驟 2 · 約 0 分鐘

2. 事前準備

必要條件

請確認已安裝下列項目：

- Go 工具鍊 1.24.5 以上版本。如需安裝操作說明，請參閱 Go 的[入門指南](#)。
- 通訊協定緩衝區編譯器 `protoc`，版本須為 3.27.1 以上。如需安裝操作說明，請參閱編譯器的[安裝指南](#)。
- Go 和 gRPC 適用的通訊協定緩衝區編譯器外掛程式。如要安裝這些外掛程式，請執行下列指令：

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

更新 `PATH` 變數，讓通訊協定緩衝區編譯器可以找到外掛程式：

```
export PATH="$PATH:$(go env GOPATH)/bin"
```

取得程式碼

為避免您必須從頭開始，本程式碼研究室提供應用程式的原始碼架構，供您完成。下列步驟將說明如何完成應用程式，包括使用通訊協定緩衝區編譯器外掛程式產生樣板 gRPC 程式碼。

首先，建立程式碼研究室工作目錄，然後 `cd` 到該目錄：

〇〇〇

```
mkdir streaming-grpc-go-getting-started && cd streaming-grpc-go-getting-started
```

下載並擷取程式碼研究室：

〇〇

```
curl -sL https://github.com/grpc-ecosystem/grpc-codelabs/archive/refs/heads/v1.tar.gz \
| tar xvz --strip-components=4 \
grpc-codelabs-1/codelabs/grpc-go-streaming/start_here
```

或者，您也可以下載只包含 Codelab 目錄的 .zip 檔案，然後手動解壓縮。

如要略過實作的輸入作業，可以在 [GitHub](#) 上取得完整的原始碼。

3. 定義訊息和服務

首先，請使用[通訊協定緩衝區](#)定義應用程式的 gRPC 服務、RPC 方法，以及要求和回應訊息類型。你的服務將提供：

- 伺服器實作且用戶端呼叫的 RPC 方法 `ListFeatures`、`RecordRoute` 和 `RouteChat`。
- 訊息類型 `Point`、`Feature`、`Rectangle`、`RouteNote` 和 `RouteSummary`，這些是呼叫上述方法時，用戶端和伺服器之間交換的資料結構。

這些 RPC 方法及其訊息類型都會在提供的原始碼 `routeguide/route_guide.proto` 檔案中定義。

通訊協定緩衝區通常稱為 protobuf。如要進一步瞭解 gRPC 術語，請參閱 gRPC 的「[核心概念](#)、[架構和生命週期](#)」。

定義訊息類型

在原始碼的 `routeguide/route_guide.proto` 檔案中，請先定義 `Point` 訊息型別。`Point` 代表地圖上的經緯度座標組合。在本程式碼研究室中，請使用整數做為座標：

```
message Point {
  int32 latitude = 1;
  int32 longitude = 2;
}
```

數字 `1` 和 `2` 是 `message` 結構中每個欄位的專屬 ID 號碼。

接著定義 `Feature` 訊息類型。`Feature` 使用 `string` 欄位，為 `Point` 指定位置的某個項目提供名稱或郵寄地址：

```
message Feature {
  // The name or address of the feature.
  string name = 1;

  // The point where the feature is located.
  Point location = 2;
}
```

接著是 `Rectangle` 訊息，代表經緯度矩形，以兩個對角點「lo」和「hi」表示。

```
message Rectangle {
  // One corner of the rectangle.
  Point lo = 1;

  // The other corner of the rectangle.
  Point hi = 2;
}
```

此外，`RouteNote` 訊息代表在特定時間點傳送的訊息。

```
message RouteNote {
    // The location from which the message is sent.
    Point location = 1;

    // The message to be sent.
    string message = 2;
}
```

我們也需要 `RouteSummary` 訊息。這則訊息是針對 `RecordRoute` RPC 收到的回應，下一節將說明這項 RPC。其中包含收到的個別點數、偵測到的特徵數量，以及涵蓋的總距離 (各點之間距離的累加總和)。

```
message RouteSummary {
    // The number of points received.
    int32 point_count = 1;

    // The number of known features passed while traversing the route.
    int32 feature_count = 2;

    // The distance covered in metres.
    int32 distance = 3;

    // The duration of the traversal in seconds.
    int32 elapsed_time = 4;
}
```

定義服務方法

如要定義服務，請在 `.proto` 檔案中指定具名服務。`route_guide.proto` 檔案具有名為 `RouteGuide` 的 `service` 結構，可定義應用程式服務提供的一或多個方法。

在服務定義中定義 `RPC` 方法，並指定要求和回應類型。在本程式碼研究室的這一節中，我們將定義：

ListFeatures

取得指定 `Rectangle` 內可用的 `Feature`。由於矩形可能涵蓋大範圍，並包含大量特徵，因此系統會串流傳回結果，而非一次傳回 (例如在含有重複欄位的訊息中)。

這個 RPC 的適當類型是 *伺服器端* 串流 RPC：用戶端會將要求傳送至伺服器，並取得串流來讀取一系列訊息。用戶端會從傳回的串流讀取訊息，直到沒有更多訊息為止。如範例所示，您可以在回應型別前加上 `stream` 關鍵字，指定伺服器端串流方法。

```
rpc ListFeatures(Rectangle) returns (stream Feature) {}
```

RecordRoute

接受所遍歷路徑上的 `Point` 串流，並在遍歷完成時傳回 `RouteSummary`。

在這種情況下，*用戶端串流* RPC 似乎很合適：用戶端會寫入一連串訊息並傳送至伺服器，同樣是使用提供的串流。用戶端寫完訊息後，會等待伺服器讀取所有訊息並傳回回應。如要指定用戶端串流方法，請在要求型別前加上串流關鍵字。

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

RouteChat

接受在路線遍歷期間傳送的 `RouteNote` 串流，同時接收其他 `RouteNote` (例如來自其他使用者)。

這正是 *雙向串流* 的適用用途。雙向串流 RPC 的兩端都會使用讀寫串流傳送一連串訊息。這兩個串流各自獨立運作，因此用戶端和伺服器可以依任何順序讀取及寫入資料。

舉例來說，伺服器可以等待接收所有用戶端訊息，再撰寫回覆內容；也可以讀取訊息，然後撰寫訊息；或是讀取和撰寫訊息的某種組合。

每個串流中的訊息順序都會保留。如要指定這類方法，請在要求和回應前加上串流關鍵字。

```
rpc RouteChat(stream RouteNote) returns (stream RouteNote) {}
```

4. 產生用戶端和伺服器程式碼

接著，使用通訊協定緩衝區編譯器，從 `.proto` 檔案產生用戶端和伺服器的樣板 gRPC 程式碼。在 `routeguide` 目錄中執行：

```
protoc --go_out=. --go_opt=paths=source_relative \  
      --go-grpc_out=. --go-grpc_opt=paths=source_relative \  
      route_guide.proto
```

這個指令會產生下列檔案：

- `route_guide.pb.go`，其中包含建立應用程式訊息類型，以及存取訊息資料和代表訊息的類型定義等函式。
- `route_guide_grpc.pb.go`，其中包含用戶端用來呼叫服務遠端 gRPC 方法的函式，以及伺服器用來提供該遠端服務的函式。

接著，我們會在伺服器端實作方法，以便在用戶端傳送要求時，伺服器可以回覆答案。

步驟 5 · 約 5 分鐘

5. 實作服務

首先，我們來看看如何建立 `RouteGuide` 伺服器。如要讓 `RouteGuide` 服務正常運作，需要完成以下兩個步驟：

- 實作從服務定義產生的服務介面：執行服務的實際「工作」。
- 執行 gRPC 伺服器，監聽來自用戶端的要求，並將要求分派至正確的服務實作。

讓我們在 `server/server.go` 中實作 `RouteGuide`。

實作 `RouteGuide`

我們需要實作產生的 `RouteGuideService` 介面。實作方式如下所示。

```
type routeGuideServer struct {  
    ...  
}  
...  
func (s *routeGuideServer) ListFeatures(rect *pb.Rectangle, stream  
pb.RouteGuide_ListFeaturesServer) error {  
    ...  
}  
...  
  
func (s *routeGuideServer) RecordRoute(stream pb.RouteGuide_RecordRouteServer) error {  
    ...  
}  
...  
  
func (s *routeGuideServer) RouteChat(stream pb.RouteGuide_RouteChatServer) error {  
    ...  
}
```

我們來詳細瞭解各項 RPC 實作。

伺服器端串流 RPC

請先從其中一個串流 RPC 開始。`ListFeatures` 是伺服器端串流 RPC，因此我們需要將多個 `Feature` 傳送回用戶端。

```
func (s *routeGuideServer) ListFeatures(rect *pb.Rectangle, stream  
pb.RouteGuide_ListFeaturesServer) error {  
    for _, feature := range s.savedFeatures {  
        if inRange(feature.Location, rect) {  
            if err := stream.Send(feature); err != nil {  
                return err  
            }  
        }  
    }  
    return nil  
}
```

如您所見，這次我們取得的是要求物件 (用戶端想在其中尋找 `Features` 的 `Rectangle`) 和特殊的 `RouteGuide_ListFeaturesServer` 物件，用來編寫回應，而不是在方法參數中取得簡單的要求和回應物件。在這個方法中，我們會填入盡可能多的 `Feature` 物件，並使用 `RouteGuide_ListFeaturesServer` 的 `Send()` 方法將這些物件寫入 `RouteGuide_ListFeaturesServer`。最後，如同簡單的 RPC，我們會傳回 `nil` 錯誤，告知 gRPC 我們已完成撰寫回應。如果此呼叫發生任何錯誤，我們會傳回非空值錯誤；gRPC 層會將其轉換為適當的 RPC 狀態，以便透過網路傳送。

用戶端串流 RPC

現在來看看稍微複雜一點的內容：用戶端串流方法 `RecordRoute`，我們會從用戶端取得 `Points` 串流，並傳回包含行程資訊的單一 `RouteSummary`。如您所見，這次方法完全沒有要求參數。而是會取得 `RouteGuide_RecordRouteServer` 串流，伺服器可使用這個串流讀取及寫入訊息，也就是使用 `Recv()` 方法接收用戶端訊息，並使用 `SendAndClose()` 方法傳回單一回應。

```
func (s *routeGuideServer) RecordRoute(stream pb.RouteGuide_RecordRouteServer) error {
    var pointCount, featureCount, distance int32
    var lastPoint *pb.Point
    startTime := time.Now()
    for {
        point, err := stream.Recv()
        if err == io.EOF {
            endTime := time.Now()
            return stream.SendAndClose(&pb.RouteSummary{
                PointCount:    pointCount,
                FeatureCount:    featureCount,
                Distance:         distance,
                ElapsedTime:       int32(endTime.Sub(startTime).Seconds()),
            })
        }
        if err != nil {
            return err
        }
        pointCount++
        for _, feature := range s.savedFeatures {
            if proto.Equal(feature.Location, point) {
                featureCount++
            }
        }
        if lastPoint != nil {
            distance += calcDistance(lastPoint, point)
        }
        lastPoint = point
    }
}
```

在方法主體中，我們會使用 `RouteGuide_RecordRouteServer` 的 `Recv()` 方法，重複讀取用戶端對要求物件的要求 (在本例中為 `Point`)，直到沒有其他訊息為止：伺服器需要在每次呼叫後檢查 `Recv()` 傳回的錯誤。如果是 `nil`，表示串流仍正常，可以繼續讀取；如果是 `io.EOF`，表示訊息串流已結束，伺服器可以傳回 `RouteSummary`。如果不是，我們會「原封不動」地傳回錯誤，以便 gRPC 層將其轉換為 RPC 狀態。

雙向串流 RPC

最後，我們來看看雙向串流 RPC `RouteChat()`。

```
func (s *routeGuideServer) RouteChat(stream pb.RouteGuide_RouteChatServer) error {
    for {
        in, err := stream.Recv()
        if err == io.EOF {
            return nil
        }
        if err != nil {
            return err
        }
        key := serialize(in.Location)

        s.mu.Lock()
        s.routeNotes[key] = append(s.routeNotes[key], in)
        // Note: this copy prevents blocking other clients while serving this one.
        // We don't need to do a deep copy, because elements in the slice are
        // insert-only and never modified.
        rn := make([]*pb.RouteNote, len(s.routeNotes[key]))
        copy(rn, s.routeNotes[key])
        s.mu.Unlock()

        for _, note := range rn {
            if err := stream.Send(note); err != nil {
                return err
            }
        }
    }
}
```

這次我們取得的 `RouteGuide_RouteChatServer` 串流，與用戶端串流範例一樣，可用於讀取和寫入訊息。不過，這次我們會在用戶端仍將訊息寫入訊息串流時，透過方法串流傳回值。這裡的讀取和寫入語法與用戶端串流方法非常相似，但伺服器會使用串流的 `send()` 方法，而不是 `SendAndClose()`，因為伺服器會寫入多個回應。雖然雙方一律會依撰寫順序收到對方的訊息，但用戶端和伺服器可以依任意順序讀取及寫入資料，因為資料串完全獨立運作。

啟動伺服器

實作所有方法後，我們也需要啟動 gRPC 伺服器，讓用戶端實際使用服務。下列程式碼片段顯示我們如何為 `RouteGuide` 服務執行這項操作：

```
lis, err := net.Listen("tcp", fmt.Sprintf("localhost:%d", *port))
if err != nil {
    log.Fatalf("failed to listen: %v", err)
}

grpcServer := grpc.NewServer()

s := &routeGuideServer{routeNotes: make(map[string][]*pb.RouteNote)}
s.loadFeatures()
pb.RegisterRouteGuideServer(grpcServer, s)
grpcServer.Serve(lis)
```

以下是 `main()` 的運作方式：

1. 使用 `lis, err := net.Listen(...)` 指定要用來監聽遠端用戶端要求的 TCP 通訊埠。根據預設，應用程式會使用變數 `port` 指定的 TCP 通訊埠 `50051`，或在執行伺服器時，透過指令列傳遞 `--port` 切換。如果無法開啟 TCP 連接埠，應用程式會以嚴重錯誤結束。
2. 使用 `grpc.NewServer(...)` 建立 gRPC 伺服器的執行個體，並將此執行個體命名為 `grpcServer`。
3. 建立 `routeGuideServer` 的指標，這是代表應用程式 API 服務的結構，並將指標命名為 `s`。
4. 使用 `s.loadFeatures()` 填入 `s.savedFeatures` 陣列。
5. 向 gRPC 伺服器註冊服務實作。
6. 在伺服器上使用我們的連接埠詳細資料呼叫 `Serve()`，以執行用戶端要求的封鎖等待作業；這項作業會持續執行，直到程序終止或呼叫 `Stop()` 為止。

函式 `loadFeatures()` 會從 `server/testdata.go` 取得座標到位置的對應。

步驟 6 · 約 5 分鐘

6. 建立用戶端

現在編輯 `client/client.go`，您將在此實作用戶端程式碼。

如要呼叫遠端服務的方法，我們首先需要建立 gRPC 通道，與伺服器通訊。方法是在用戶端的 `main()` 函式中，將伺服器的目標 URI 字串 (在本例中，這只是位址和連接埠號碼) 傳遞至 `grpc.NewClient()`，如下所示：

```
// Set up a connection to the gRPC server.
conn, err := grpc.NewClient("dns:///"+*serverAddr,
    grpc.WithTransportCredentials(insecure.NewCredentials()))
if err != nil {
    log.Fatalf("fail to dial: %v", err)
}
defer conn.Close()
```

伺服器位址 (由變數 `serverAddr` 定義) 預設為 `localhost:50051`，執行用戶端時，可透過指令列的 `--addr` 切換鍵覆寫。

如果用戶端需要連線至需要驗證憑證的服務 (例如 TLS 或 JWT 憑證)，用戶端可以將 `DialOptions` 物件做為參數傳遞至 `grpc.NewClient`，其中包含必要憑證。`RouteGuide` 服務不需要任何憑證。

設定 gRPC 通道後，我們需要用戶端虛設常式，才能透過 Go 函式呼叫執行 RPC。我們使用應用程式 `.proto` 檔案產生的 `route_guide_grpc.pb.go` 檔案提供的 `NewRouteGuideClient` 方法，取得該存根。

```
import (pb "github.com/grpc-ecosystem/codelabs/getting_started_streaming/routeguide")

client := pb.NewRouteGuideClient(conn)
```

呼叫服務方法

接著來看看如何呼叫服務方法。在 gRPC-Go 中，RPC 會以封鎖/同步模式運作，也就是說，RPC 呼叫會等待伺服器回應，並傳回回應或錯誤。

伺服器端串流 RPC

我們在此呼叫伺服器端串流方法 `ListFeatures`，該方法會傳回地理 `Feature` 物件的串流。

```

rect := &pb.Rectangle{ ... } // initialize a pb.Rectangle
log.Printf("Looking for features within %v", rect)
stream, err := client.ListFeatures(context.Background(), rect)
if err != nil {
    log.Fatalf("client.ListFeatures failed: %v", err)
}
for {
    // For server-to-client streaming RPCs, you call stream.Recv() until it
    // returns io.EOF.
    feature, err := stream.Recv()
    if err == io.EOF {
        break
    }
    if err != nil {
        log.Fatalf("client.ListFeatures failed: %v", err)
    }
    log.Printf("Feature: name: %q, point:(%v, %v)", feature.GetName(),
        feature.GetLocation().GetLatitude(), feature.GetLocation().GetLongitude())
}

```

如同簡單的 RPC，我們將內容和要求傳遞給方法。不過，我們傳回的不是回應物件，而是

`RouteGuide_ListFeaturesClient` 的例項。用戶端可以使用 `RouteGuide_ListFeaturesClient` 串流讀取伺服器的回應。我們會使用 `RouteGuide_ListFeaturesClient` 的 `Recv()` 方法，重複讀取伺服器對回應通訊協定緩衝區物件 (在本例中為 `Feature`) 的回應，直到沒有其他訊息為止：用戶端需要在每次呼叫後檢查 `Recv()` 傳回的錯誤 `err`。如果是 `nil`，表示串流仍正常，可以繼續讀取；如果是 `io.EOF`，表示訊息串流已結束；否則必須是 RPC 錯誤，並透過 `err` 傳遞。

用戶端串流 RPC

用戶端串流方法 `RecordRoute` 與伺服器端方法類似，差別在於我們只將內容傳遞至方法，並取得 `RouteGuide_RecordRouteClient` 串流，可用於寫入及讀取訊息。

```

// Create a random number of random points
r := rand.New(rand.NewSource(time.Now().UnixNano()))
pointCount := int(r.Int31n(100)) + 2 // Traverse at least two points
var points []*pb.Point
for i := 0; i < pointCount; i++ {
    points = append(points, randomPoint(r))
}
log.Printf("Traversing %d points.", len(points))
c2sStream, err := client.RecordRoute(context.TODO())
if err != nil {
    log.Fatalf("client.RecordRoute failed: %v", err)
}
// Stream each point to the server.
for _, point := range points {
    if err := c2sStream.Send(point); err != nil {
        log.Fatalf("client.RecordRoute: stream.Send(%v) failed: %v", point, err)
    }
}
// Close the stream and receive the RouteSummary from the server.
reply, err := c2sStream.CloseAndRecv()
if err != nil {
    log.Fatalf("client.RecordRoute failed: %v", err)
}
log.Printf("Route summary: %v", reply)

```

`RouteGuide_RecordRouteClient` 具有 `Send()` 方法，可用於將要求傳送至伺服器。使用 `Send()` 將用戶端要求寫入串流後，我們需要在串流上呼叫 `CloseAndRecv()`，讓 gRPC 知道我們已完成寫入，並預期會收到回應。我們是從 `CloseAndRecv()` 傳回的 `err` 取得 RPC 狀態。如果狀態為 `nil`，則 `CloseAndRecv()` 的第一個傳回值會是有效的伺服器回應。

雙向串流 RPC

最後，我們來看看雙向串流 RPC `RouteChat()`。如同 `RecordRoute`，我們只會將內容物件傳遞至方法，並取得可用於寫入和讀取訊息的串流。不過，這次我們會在伺服器仍將訊息寫入訊息串流時，透過方法串流傳回值。

```

biDiStream, err := client.RouteChat(context.Background())
if err != nil {
    log.Fatalf("client.RouteChat failed: %v", err)
}
// this channel is used to wait for the receive goroutine to finish.
recvDoneCh := make(chan struct{})
// receive goroutine.
go func() {
    for {
        in, err := biDiStream.Recv()
        if err == io.EOF {
            // read done.
            close(recvDoneCh)
            return
        }
        if err != nil {
            log.Fatalf("client.RouteChat failed: %v", err)
        }
        log.Printf("Got message %s at point(%d, %d)", in.Message, in.Location.Latitude,
in.Location.Longitude)
    }
}()
// send messages simultaneously.
for _, note := range notes {
    if err := biDiStream.Send(note); err != nil {
        log.Fatalf("client.RouteChat: stream.Send(%v) failed: %v", note, err)
    }
}
biDiStream.CloseSend()
// wait for the receive goroutine to finish.
<-recvDoneCh

```

這裡的讀取和寫入語法與用戶端串流方法非常相似，但我們會在呼叫完成後使用串流的 `CloseSend()` 方法。雖然雙方一律會依撰寫順序收到對方的訊息，但用戶端和伺服器可以依任意順序讀取及寫入資料，因為資料串完全獨立運作。

7. 立即試用

在應用程式的工作目錄中執行下列指令，確認伺服器 and 用戶端可正常運作：

1. 在一個終端機中執行伺服器：

```
cd server
go run .
```

2. 從另一個終端機執行用戶端：

```
cd client
go run .
```

您會看到類似下列的輸出內容，為求清楚起見，時間戳記已省略：

```
Looking for features within lo:<latitude:400000000 longitude:-750000000 > hi:
<latitude:420000000 longitude:-730000000 >
name:"Patriots Path, Mendham, NJ 07945, USA" location:<latitude:407838351
longitude:-746143763 >
...
name:"3 Hasta Way, Newton, NJ 07860, USA" location:<latitude:410248224
longitude:-747127767 >
Traversing 56 points.
Route summary: point_count:56 distance:497013163
Got message First message at point(0, 1)
Got message Second message at point(0, 2)
Got message Third message at point(0, 3)
Got message First message at point(0, 1)
Got message Fourth message at point(0, 1)
Got message Second message at point(0, 2)
Got message Fifth message at point(0, 2)
Got message Third message at point(0, 3)
Got message Sixth message at point(0, 3)
```

步驟 8 · 約 0 分鐘

8. 後續步驟

- 如要瞭解 gRPC 的運作方式，請參閱「[gRPC 簡介](#)」和「[核心概念](#)」。
 - 逐步完成[基礎教學課程](#)。
 - 探索 [API 參考資料](#)。
-